

REPORTE DE PRÁCTICA: ARREGLOS DE CARACTERES (STRINGS) Y SU APLICACIÓN AVANZADA

Asignatura: Programación Estructurada
Estudiante: Aldo Ezequiel Rodriguez Godoy
Código: 224097617
Sección: D04
Profesor: JORGE ERNESTO LOPEZ ARCE DELGADO
Fecha: 13 de marzo del 2026

INTRODUCCIÓN

En el ámbito del lenguaje C, los textos o cadenas de caracteres no existen como un tipo de dato primitivo unificado. En su lugar, se implementan a través de arreglos de caracteres unidimensionales, donde cada casilla de memoria almacena un único carácter bajo el estándar de codificación ASCII. La característica semántica fundamental de estas estructuras es que el final del texto se encuentra explícitamente delimitado por el carácter especial nulo, representado mediante la secuencia de escape `"\0"`. Este carácter terminador le indica de manera unívoca a las funciones de impresión y lectura los límites reales del procesamiento informático.

La manipulación estándar de cadenas en C requiere la inclusión del encabezado `<string.h>`, el cual dota al desarrollador de funciones esenciales para copiar bloques lineales de texto mediante ``strcpy``, concatenar cadenas con ``strcat``, calcular longitudes físicas usando ``strlen`` o ejecutar validaciones de equivalencia a través de ``strcmp``. Asimismo, se prefiere el uso de la función ``fgets`` sobre el tradicional ``scanf`` para la captura interactiva de cadenas en consola, dado que esta última trunca la entrada al encontrar el primer carácter de espacio en blanco, mientras que ``fgets`` lee de forma íntegra líneas completas y previene desbordamientos de búfer mediante la parametrización de su tamaño físico.

En este ejercicio de laboratorio, realizamos una extensión de gran complejidad para nuestro simulador de "Batalla Naval". El software fue transformado a un modelo de juego multijugador con turnos alternados dinámicos. Para lograr este objetivo de diseño, el código integra estructuras multidimensionales avanzadas que cruzan tanto arreglos enteros bidimensionales para controlar los estados físicos del tablero, como un arreglo tridimensional de tipo ``char`` que asocia cadenas de identificación textual (nombres de barcos personalizados) a coordenadas geográficas específicas.

OBJETIVO

Objetivo General

Construir, estructurar e implementar una versión extendida multijugador del juego de Batalla Naval en lenguaje C, aplicando de forma exhaustiva la manipulación de arreglos de caracteres y funciones de la biblioteca básica `<string.h>` para la personalización de la experiencia de juego.

Objetivos Particulares

- Implementar capturas de texto seguras mediante ``fgets`` y eliminar secuencias de salto de línea con ``strcspn`` para procesar de forma correcta los nombres de los dos usuarios participantes.
- Diseñar una estructura de datos multidimensional de caracteres de cuatro dimensiones para mapear nombres textuales individuales (Fragata, Destructor, Submarino) a las posiciones geográficas de cada jugador.
- Programar un algoritmo de alternancia de turnos dinámico basado en operaciones de aritmética modular y evaluar condiciones de control de juego para validar disparos repetidos o celdas fuera de rango.

DESARROLLO

La adaptación modular del simulador de juego se programó en el archivo principal ``main.c``, implementando de forma estricta los requerimientos formales de la práctica a través de las siguientes fases secuenciales:

1. Configuración de Variables y Estructuras Multidimensionales

Declaré un arreglo de caracteres bidimensional ``char jugadores[2][20]`` para almacenar el nombre de los dos participantes. Para asignar las cadenas de texto correspondientes al armamento, desarrollé una matriz de caracteres de cuatro dimensiones (``char barcos[2][5][5][15]``), donde la primera dimensión indica el índice del jugador propietario, la segunda la fila, la tercera la columna y la cuarta el tamaño del búfer de caracteres asignado al nombre de la embarcación. Asimismo, se generó un tablero tridimensional de enteros (``int tablero[2][5][5]``) para gestionar de forma paralela los estados físicos numéricos del juego (0: mar, 1: barco oculto, 2: agua atacada, 3: impacto).

2. Captura de Cadenas e Inicialización Estructural

Mediante ``fgets(jugadores[idx], sizeof(...), stdin)``, el sistema solicita al inicio del programa la entrada textual de los nombres de los contrincantes. Dado que ``fgets`` almacena el salto de línea generado al presionar la tecla Enter, utilicé la función ``strcspn`` combinada con el carácter nulo (``""``) para limpiar la cadena de forma segura. Posteriormente, mediante ciclos ``for`` anidados triples, inicialicé las celdas numéricas en ``0`` y copié cadenas vacías a las matrices de nombres utilizando la función ``strcpy`` de la biblioteca de C.

3. Posicionamiento de Flotas Personalizadas

Utilizando funciones de asignación directa de strings, coloqué de manera simétrica tres barcos para cada jugador, enlazando el estado lógico numérico en la matriz ``tablero`` y copiando el nombre del barco correspondiente en la matriz ``barcos`` mediante ``strcpy``:

- **Jugador 1:** Fragata en (1,2), Destructor en (3,4) y Submarino en (0,0).
- **Jugador 2:** Fragata en (2,1), Destructor en (4,3) y Submarino en (0,4).

4. Mecánica Lógica de Turnos Alternados y Evaluación de Impactos

El núcleo del software opera dentro de un ciclo condicional ``while`` que se mantiene activo en tanto el arreglo de contadores ``barcos_restantes`` para ambos jugadores sea estrictamente mayor a cero. En cada ciclo, se determina dinámicamente un índice ``atacante`` y un ``defensor`` usando operaciones

modulares (``turno % 2``). El sistema imprime de forma personalizada el nombre del atacante y renderiza visualmente el estado del mapa del defensor ocultando las embarcaciones vivas.

Al procesar el disparo mediante coordenadas ingresadas con ``scanf``, el algoritmo evalúa de forma jerárquica: validación de límites (0 a 4) y control de disparos duplicados (estados 2 o 3). Si la casilla contiene el estado ``1``, se recupera directamente el nombre del barco almacenado en la cadena correspondiente mediante ``barcos[defensor][fila][columna]``, imprimiendo un mensaje enriquecido en el que se especifica qué barco fue alcanzado y reduciendo el contador de flota del defensor. Cuando este contador llega a cero, el ciclo se rompe y se proclama al ganador.

CONCLUSIÓN

La realización de esta práctica de laboratorio representó un salto de calidad enorme en el manejo de estructuras de datos en C. Trabajar con cadenas de caracteres me obligó a dejar atrás la lógica simple de otros lenguajes de programación y comprender que en C los textos requieren un control minucioso de la memoria, asegurando siempre el uso del carácter nulo terminador ``"` para evitar lecturas de datos basura.

Lo más complejo e instructivo de la actividad fue diseñar e implementar la matriz de caracteres de cuatro dimensiones para mapear los nombres de los barcos. Aprender a coordinar en un mismo punto lógico el índice del jugador, las coordenadas bidimensionales del plano de juego y el búfer de texto mediante la función ``strcpy`` me dio un entendimiento total sobre el manejo de arreglos multidimensionales. Asimismo, la limpieza de las cadenas con ``strcspn`` y el control interactivo de los turnos alternados mediante operaciones de residuo numérico me demostraron cómo construir código robusto y amigable para el usuario. Esta práctica consolida todas las bases de control de flujo, arreglos y cadenas, preparándome de forma excelente para abordar temas avanzados como estructuras de datos compuestas (structs) y el uso de punteros en los siguientes módulos de la materia.

REFERENCIAS

- Kernighan, B. W., & Ritchie, D. M. (1988). *The C Programming Language* (2nd ed.). Prentice Hall.
- Prata, S. (2013). *C Primer Plus* (6th ed.). Addison-Wesley Professional.